

CS116 · ĐỀ TÀI T11 · UIT, ĐHQG-HCM

Hệ thống đọc hiểu và trả lời câu hỏi tiếng Việt

Vietnamese Extractive Machine Reading Comprehension
trên UIT-ViQuAD 2.0

Bốn mô hình · 500 câu kiểm định · mọi con số sinh từ một lần chạy thật

LỘ TRÌNH 15 PHÚT

Bài toán

Dữ liệu

Kiến trúc

Kết quả

Ba phát hiện

Demo

NHÓM 7

Lê Quang Thi
25210337

Trần Trọng Tấn
25210334

Nguyễn Quang Lâm
25210289

Vỏ Cẩm Thu
25210342

GVHD

ThS. Nguyễn Hữu Quyền

Trích xuất, không phải sinh

ĐOẠN VĂN (CONTEXT)

Hà Nội là thủ đô của nước Cộng hoà Xã hội chủ nghĩa Việt Nam. Hà Nội được UNESCO công nhận là Thành phố vì hoà bình vào **năm 1999**.

CÂU HỎI

Hà Nội được UNESCO công nhận vào năm nào?

Đáp án: năm 1999

**Mô hình không sinh chữ mới.
Nó chỉ ra vị trí.**

Vì đáp án luôn nằm trong context, ta có một bất biến kiểm được bằng máy:

$\text{predict}(\text{ctx}, q) \subset \text{ctx}$

và nó được kiểm thử cho mọi mô hình.

DATASET

UIT-ViQuAD 2.0

ĐỊNH DẠNG

SQuAD-2.0

METRIC

Exact Match + token-F1

ĐẦU RA

(start_char, end_char)

Tập test là blind set — không chấm được

ĐO TRỰC TIẾP TỪ FILE TẢI VỀ

SPLIT	CÂU HỎI	CONTEXT	ARTICLE	IMPOSSIBLE	CHẤM ĐƯỢC
train	28.454	4.101	138	9.216	28.454
validation	3.814	557	19	1.161	3.814
test	7.301	1.241	48	0	0

7.301 câu test đều có gold rỗng, đồng thời `is_impossible = False`.

Hai điều đó mâu thuẫn nhau nếu coi là nhãn thật ⇒ đáp án đã bị lược bỏ để dùng cho leaderboard.

Chấm nhầm trên tập này → EM 100%.

Bạn em chặn bằng code, không bằng lời hứa

`assert_gradeable(examples)`

ném `ValueError` trước MỌI lần chấm điểm, thay vì để một mô hình luôn trả rỗng đạt EM 100%.

30,4%

câu validation là impossible — metric tổng trộn hai kỹ năng

4.101 ≠ 138

context không phải article: nhầm hai đơn vị làm hỏng chống leakage

Bốn hệ thống — mỗi cái trả lời một câu hỏi khác nhau

1 TF-IDF Baseline

không huấn luyện

Bài toán này giải được bằng so khớp từ khoá thuần không?

2 XLM-R squad2

zero-shot

Transfer từ SQuAD-2.0 tiếng Anh sang tiếng Việt được bao nhiêu?

3 mBERT + QA

fine-tuned · MPS

Fine-tune in-domain thêm được bao nhiêu, với encoder đa ngữ?

4 ViSoBERT + QA

fine-tuned · MPS

Pretraining chuyên tiếng Việt đóng góp bao nhiêu?

Giả thuyết đăng ký TRƯỚC khi chạy (results/hypotheses.md) dự đoán encoder tiếng Việt sẽ thắng. Kết quả ngược lại — và được báo cáo nguyên vẹn.

Bốn tầng, một chiều phụ thuộc

Tầng dữ liệu

data.py · tagging.py

parse SQuAD-2.0 → Example · dedup & split theo CONTEXT ·
assert_no_leakage · assert_gradeable

Tầng mô hình

predictor.py · baseline_tfidf.py · transformer_qa.py ·
windowing.py · features.py · training.py

một interface duy nhất: Predictor.predict(ctx, q) -> str

Tầng đánh giá

normalize.py · metrics.py · evaluate.py

EM + token-F1 · tách answerable / impossible · provenance · cờ unreliable khi n
< 30

Tầng trình bày

src/demo/ (hàm thuần) → app/ (chỉ gọi widget) ·
make_figures.py · make_report.py

CHỈ ĐỌC results/ — không bao giờ sinh ra con số

app/ → src/demo/ → src/mrc/ → results/, data/raw/ · một mũi tên, một chiều, không ngoại lệ

Bốn bất biến — và cơ chế thực thi bằng máy

1

Extractive

Mọi `predict()` trả về chuỗi con của context

CƠ CHẾ THỰC THI

Test cho mọi Predictor; `decode_span()` ném `ValueError` nếu khoảng không hợp lệ

2

Không leakage

Một context chỉ thuộc một split

CƠ CHẾ THỰC THI

`assert_no_leakage()` chạy trong mọi đường split và làm FAIL cả run

3

Truy vết được

Mọi kết quả mang `commit`, `timestamp`, `device`, `split`, `n`

CƠ CHẾ THỰC THI

`run_evaluation()` sinh các trường này — không đường nào tạo ra kết quả thiếu chúng

4

Có `n` kèm số

Nhóm `count` < 30 tự gán `unreliable: true`

CƠ CHẾ THỰC THI

`breakdown()` gán cờ; không bảng nào trong báo cáo có số thiếu `n`

Bất biến thứ năm: demo không sinh ra con số nào — và điều đó do kernel bắt buộc, vì `models/`, `data/`, `results/` được mount read-only.

Hai quyết định kỹ thuật đã đổi cả hệ thống

ADR-003

Tự cài cửa sổ trượt

return_overflowing_tokens của transformers 5.17.0 sinh tối đa 2 cửa sổ, bất kể context dài bao nhiêu:

ĐỘ DÀI CONTEXT	THỰC TẾ	ĐÚNG RA
210 token	2	2
420 token	2	5
700 token	2	8
1400 token	2	16

Đuôi context bị cắt ÂM THẨM: không exception, không cảnh báo.

ADR-004 · ADR-005

Dấu tiếng Việt là ràng buộc cứng

PhoBERT không có fast tokenizer ⇒ không có offset_mapping ⇒ cách duy nhất lấy lại chuỗi là tokenizer.decode().

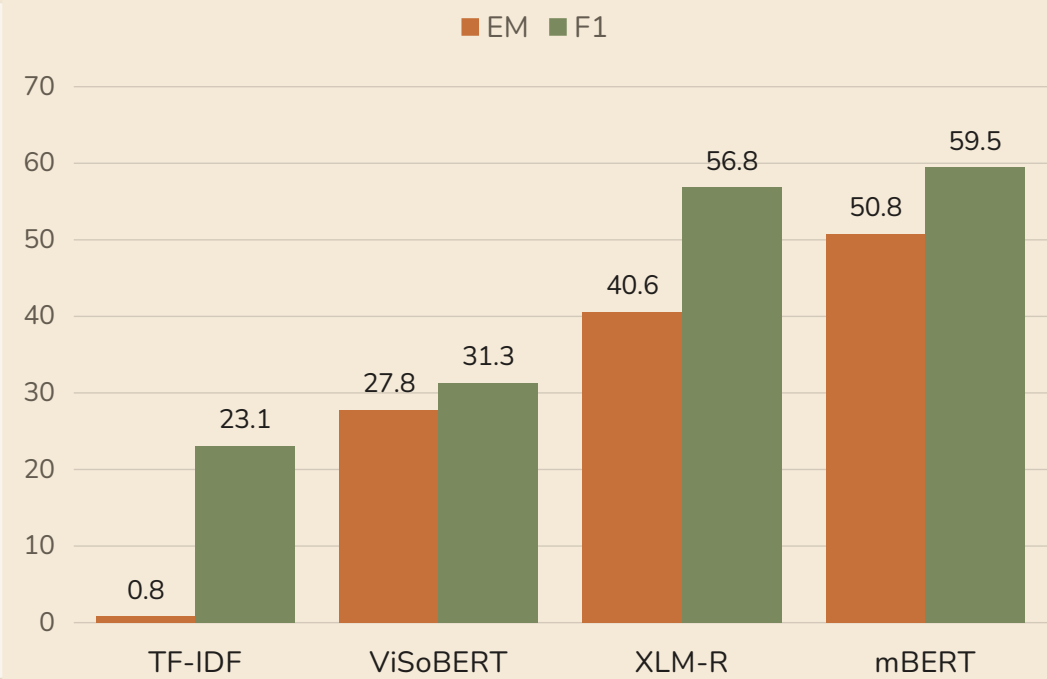
decode() làm mất dấu: "hoà" → "hoa"

Sai một dấu là sai cả EM lẫn bất biến substring. Nên hệ thống cắt thẳng chuỗi gốc theo offset ký tự — và TransformerQA TỪ CHỐI khởi tạo mô hình không có fast tokenizer.

⇒ Thay PhoBERT bằng uitnlp/visobert

UIT-ViQuAD 2.0 · validation · n = 500 · thiết bị MPS

MODEL	EM	F1	LATENCY
TF-IDF Baseline	0,80	23,09	0,54 ms
ViSoBERT + QA (fine-tuned)	27,80	31,31	22,73 ms
XLM-R squad2 (zero-shot)	40,60	56,84	14,35 ms
mBERT + QA (fine-tuned)	50,80	59,49	12,26 ms



50,80

EM cao nhất — mBERT fine-tuned

±4,3

khoảng tin cậy 95% ở n = 500

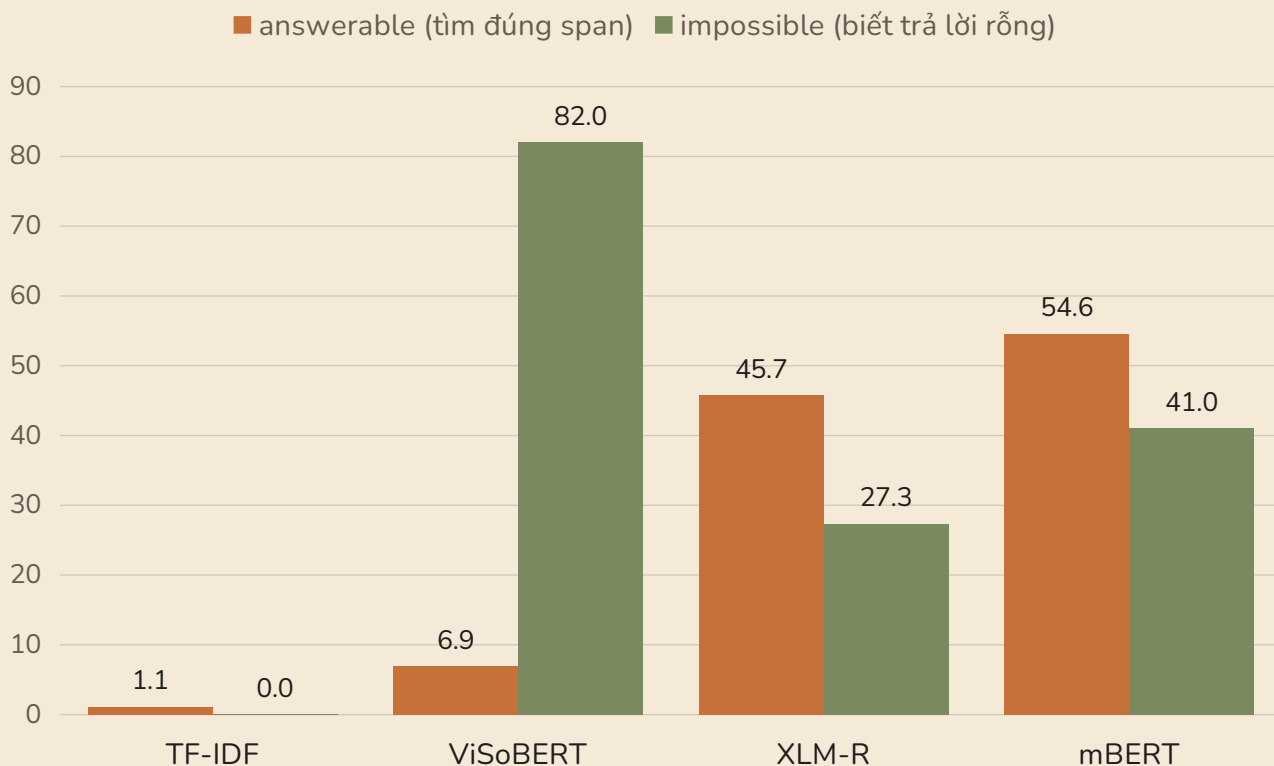
27,80

% câu impossible trong mẫu

4

commit truy vết được cho 4 kết quả

mBERT thắng KHÔNG phải vì tìm span giỏi hơn



Trên câu answerable, XLM-R zero-shot thực ra TỐT HƠN

F1 68,20 so với 66,60

— nhưng chênh lệch 1,60 điểm này nằm gọn trong nhiễu, nên không phải kết luận vững.

mBERT thắng tổng thể vì biết khi nào KHÔNG nên trả lời

impossible EM 41,01 so với 27,34

+13,67 điểm $\pm$ 11,03 — có ý nghĩa thống kê. Với ~30% câu là unanswerable, chính kỹ năng thứ hai quyết định bảng xếp hạng.

Đây là lý do báo cáo tách answerable_only và impossible_only — con số tổng trộn hai kỹ năng và che mất điều này.

ViSoBERT suy sụp về “luôn trả lời rỗng”

EM tổng 27,80

=

tỉ lệ impossible của mẫu 27,80%

Hai con số trùng nhau không phải trùng hợp.

BÓC TÁCH RA

82,01

impossible EM — gần như chỉ ăn điểm từ việc từ chối trả lời

6,93

answerable EM — nó hầu như không tìm được span nào

DẤU HIỆU XUẤT HIỆN NGAY TRÊN ĐƯỜNG CONG HUẤN LUYỆN

VISOBERT	TRAIN LOSS	VAL EM	VAL F1	CHẨN ĐOÁN
epoch 1	3,0512	25,67	25,67	EM = F1 → suy sụp
epoch 2	2,5678	25,33	25,61	EM ≈ F1 → suy sụp
epoch 3	2,1908	27,00	30,48	bắt đầu thoát ra

F1 cho điểm bán phần nên bình thường phải cao hơn EM — khi hai metric trùng nhau, training.py tự nhận ra bằng hằng số `COLLAPSE_TOLERANCE = 0.5`.

Vì sao ViSoBERT suy sụp — ca lỗi được chẩn đoán

BỐN YẾU TỐ ĐỦ ĐỂ GIẢI THÍCH SỰ SUY SỤP

YẾU TỐ	BẰNG CHỨNG
max_length = 384 chưa chỉnh	CHƯA bù trừ — mạnh nhất
loss chưa hội tụ	2,19 sau 3 epoch (mBERT 1,30 sau 2)
sức chứa nhỏ hơn 45%	97,0M so với 177,3M tham số
mất cân bằng lớp	32,39% impossible ⇒ hố “luôn trả rỗng”
max_answer_len ĐÃ bù (30→64). Trục chưa bù là max_length — hai tham số rất dễ nói lẫn.	

NGUYÊN NHÂN: TỪ VỰNG NHỎ CHIA VỤN TIẾNG VIỆT

	MBERT	VISOBERT
vocab	119.547	15.002
"Hà Nội là thủ đô..."	17 token	23 token
context val. trung bình	204,6	326,5
token / từ	1,22	1,95
context > 357 token	16 / 557	154 / 557

Chưa kết luận được rằng tiền huấn luyện tiếng Việt không giúp ích.

Từ vựng nhỏ chia vụn văn bản Wikipedia, và ngân sách cửa sổ không được chỉnh theo — đó là lời giải thích PHÙ HỢP với bằng chứng, chưa phải nhân quả đã chứng minh. ViSoBERT chưa được huấn luyện lại sau chẩn đoán, nên 27,80 EM phải đọc như một lần huấn luyện thất bại, không phải một phép đo năng lực encoder.

Kiến trúc trả về gì: test một giây, một lệnh để chấm

423

test chạy trong ~1 giây

Không tải mô hình, không cần mạng. Có được là nhờ 9/12 module trong src/mrc và toàn bộ src/demo là hàm thuần.

474

test đầy đủ, 20 tập

Bộ đầy đủ có cả 51 test tải mô hình thật. tests/test_docker.py chạy BÊN TRONG container và tự kiểm chính môi trường đang chạy nó.

2,48 GB

ảnh Docker (nén 527 MB)

Từ 10,2 GB. Ép PyTorch bản CPU trên MỌI kiến trúc — wheel aarch64 cũng kéo theo 4,1 GB thư viện CUDA.

NGƯỜI CHẤM CHỈ CẦN DOCKER — MỘT ẢNH, BỐN SERVICE

`docker compose run --rm tests`

`docker compose up app`

`./run.sh`

423 test, ~1 giây, HF_HUB_OFFLINE=1

demo tại <http://localhost:8501>

nạp ảnh đã docker save rồi mở demo

Sáu màn hình — mỗi màn hình một URL

● Hỏi đáp

/

nhập câu hỏi, kéo ngưỡng, xem span + bằng chứng

● Kết quả

/ket-qua

bảng EM/F1, breakdown, provenance

● Phân tích lỗi

/phan-tich-loi

pred (terracotta) so với gold (sage)

● Dữ liệu

/du-lieu

thống kê split, phân phối độ dài

● So sánh model

/so-sanh

bốn mô hình cạnh nhau, ô tốt nhất được TÍNH

● Huấn luyện

/huan-luyen

đường cong loss, EM, F1

Demo không sinh ra con số nào.

Mọi số trên màn hình đọc từ results/*.json hoặc đo trực tiếp từ data/raw/ qua một điểm truy cập duy nhất — kể cả thứ hạng “tốt nhất” và ô in đậm trong bảng so sánh, chúng được TÍNH chứ không gõ tay. Huấn luyện lại là màn hình tự nói đúng.

Điều bọn em nói rõ trong báo cáo

— **PhoBERT chưa từng được chạy**

Giả thuyết trung tâm “encoder tiếng Việt vượt mBERT” CHƯA KIỂM ĐƯỢC — ViSoBERT là mô hình thay thế, không phải phép kiểm

— **ViSoBERT chưa huấn luyện lại sau chẩn đoán**

Lời giải thích max_length vẫn là giả thuyết phù hợp bằng chứng, chưa phải nhân quả

— **Đánh giá trên validation, không phải test**

Test là blind split — giới hạn của dataset, không phải của thiết kế

— **n = 500, một seed duy nhất**

CI 95% khoảng $\pm 4,3$ điểm — chênh lệch F1 answerable mBERT/XLM-R KHÔNG có ý nghĩa

— **question_type là heuristic tự gán**

Ngưỡng 0.6 chọn theo quan sát — không phải nhãn có sẵn của ViQuAD

BƯỚC TIẾP THEO

Huấn luyện lại ViSoBERT với max_length 768 và lr 3e-5 — phép kiểm trực tiếp · chạy --full trên 3.814 câu · thêm epoch 3–4 cho mBERT · tích hợp PhoBERT

KẾT LUẬN

Điểm tổng trộn hai kỹ năng: tìm span, và biết khi nào im lặng.

Còn câu hỏi “tiếng Việt chuyên biệt có giúp không?” thì vẫn để ngỏ — PhoBERT chưa chạy, lần huấn luyện ViSoBERT đã suy sụp. Nói rõ điều đó cũng là một kết quả.

50,80

EM · mBERT fine-tuned

4

mô hình, một interface

423

test trong ~1 giây

0

con số được gõ tay

Cảm ơn thầy và các bạn đã lắng nghe.

[docs/ARCHITECTURE.md](#) · [docs/SYSTEM_DESIGN.md](#)